

MIDI Protocol Reference

Wire format, message set, file format,
and what PhytoScope actually sends

Bretagne Namasté — PhytoScope 1.5.1 — 2026-09-22
Written for this project. MIT licensed.

This is a **working reference**, not a specification.
The normative documents are listed in section 13; we do not
redistribute them, and where this document and they disagree,
they are right.

Contents

1 — Scope, and what this document is not	3
2 — Transport: the 31.25 kbaud current loop, and USB	4
3 — Byte structure, and running status	5
4 — Channel Voice messages	7
5 — Control Change, and Channel Mode	8
6 — Registered and non-registered parameters	10
7 — System messages	11
8 — Timing and synchronization	13
9 — Standard MIDI Files	14
10 — General MIDI	16
11 — MIDI 2.0, UMP and MIDI-CI	18
12 — What PhytoScope emits, and what it does not	19
13 — Pitfalls that cost real time	21
14 — Normative references	22

1 — Scope, and what this document is not

PhytoScope turns the electrical activity of a plant into sound. One of its outputs is MIDI: the software maps detected events onto notes and sends them to a synthesizer, a digital audio workstation, or the PhytoSense board's own hardware output. Getting that right means knowing the protocol at the byte level, because the failures are silent — a note that never stops, a channel off by one, a running-status stream that a receiver parses as garbage.

This document is the reference we needed while writing `phytoscope/music/midi_out.py` and the firmware's direct MIDI path. It covers MIDI 1.0 in the detail required to read a byte stream by hand, the Standard MIDI File container, General MIDI as far as it constrains a sender, and MIDI 2.0 at the level of what changes and why.

WHAT THIS IS NOT

It is **not normative**. The MIDI Association's specifications are, and they are copyrighted: we do not redistribute them, and section 14 says where to get them. Everything here was written to be checked against them. Where it is wrong, they win.

Numbers are written `0x90` for hexadecimal and plain for decimal. Bit 7 is the most significant bit of a byte. "Channel 1" means what a musician calls channel 1, which is the nibble `0x0` on the wire — the off-by-one that section 12 returns to.

2 — Transport: the 31.25 kbaud current loop, and USB

2.1 — The original electrical layer

MIDI 1.0's wire layer is an asynchronous serial link, deliberately crude so that it could be built in 1983 from parts anyone had:

Property	Value
Bit rate	31,250 bits/s, $\pm 1\%$
Framing	1 start bit, 8 data bits, 1 stop bit, no parity (8N1)
Bits per byte on the wire	10
Time per byte	320 μs
Signaling	5 mA current loop, opto-isolated at the input
Connector	5-pin DIN 41524 (later also 3.5 mm TRS, type A)
Direction	One way. An IN and an OUT are separate cables.

On the 5-pin DIN, pin 4 sources the current, pin 5 sinks it, and pin 2 is the shield — connected at the OUT end only, which is what keeps a ground loop out of the audio. Pins 1 and 3 are unused in MIDI 1.0. The input is opto-isolated precisely so that two pieces of equipment on different mains circuits cannot hum at each other through the MIDI cable.

WHY 31.25 KBAUD

It is 1 MHz divided by 32, which any 1980s microcontroller could derive from its own clock without a dedicated crystal. The consequence is the one that still matters: **320 μs per byte**. A three-byte Note On takes about 1 ms, so a ten-note chord takes about 10 ms to leave the port, and the last note of that chord is measurably late. This is not a defect to be fixed; it is the budget you work within.

2.2 — USB-MIDI

A USB device that declares the Audio class with a MIDISStreaming subclass carries the same messages, packed into 4-byte USB-MIDI Event Packets: one byte of Cable Number and Code Index Number, then up to three bytes of MIDI. The message set does not change. What changes is the timing: USB delivers in bursts on frame boundaries, so inter-message jitter is different in character — generally smaller in total, but lumpier — than on the current loop.

PhytoSense exposes a composite USB device; the MIDI path, when enabled, sits alongside the measurement stream and the serial control port. The software's own MIDI output goes through the host operating system's MIDI stack (ALSA, CoreMIDI, or Windows MM) and never touches USB directly.

3 — Byte structure, and running status

3.1 — Status bytes and data bytes

Every byte is one or the other, and bit 7 decides which. This single rule is what makes a MIDI stream resynchronizable: a receiver that joins mid-flight can discard bytes until it sees one with bit 7 set, and it is then aligned.

Kind	Range	Bit 7	Carries
Status byte	0x80–0xFF	1	What the message is, and (for channel messages) which channel
Data byte	0x00–0x7F	0	7 bits of payload — a note number, a velocity, a controller value

A channel message's status byte splits into two nibbles: the high nibble is the message type, the low nibble is the channel, 0–15, displayed to musicians as 1–16.

Status byte of a channel message

```

bit   7  6  5  4  3  2  1  0
      1  t  t  t  c  c  c  c
        \  /   \  /
        type   channel 0-15 (shown as 1-16)

```

Example: 0x93 = 1001 0011 = Note On, channel 4

3.2 — Running status

A sender may omit the status byte when it equals the previous one. The receiver keeps the last channel status it saw and applies it to any data bytes that follow. A stream of eighty notes on one channel therefore costs 161 bytes rather than 240 — a 33% saving on a 31.25 kbaud link, which is why the feature exists.

Without running status	With running status
90 3C 40	90 3C 40
90 3E 40	3E 40
90 40 40	40 40
80 3C 00	80 3C 00
80 3E 00	3E 00

Three rules govern it, and every one of them has bitten somebody:

1. **Only channel messages** (0x80 – 0xEF) participate. A System Common message *clears* the running status.
2. **System Real-Time messages do not clear it.** A timing clock (0xF8) can appear between two data bytes of a running-status message, and the receiver must step over it and carry on.
3. **A receiver must accept it; a sender need not use it.** Emitting full status bytes is always legal, and is what PhytoScope does.

THE CLASSIC PARSER BUG

A parser that resets running status on *any* byte with bit 7 set will mis-handle rule 2: it drops the rest of the note whenever a clock arrives mid-message. The stream is valid; the parser is not. If you write a MIDI input, test it with clock running.

4 — Channel Voice messages

Seven message types, each addressed to one of sixteen channels. This table is the core of the protocol; everything else is around it.

Status	Message	Data byte 1	Data byte 2	Bytes
0x8n	Note Off	Key number 0–127	Release velocity 0–127	3
0x9n	Note On	Key number 0–127	Velocity 1–127; 0 means Note Off	3
0xA _n	Polyphonic Key Pressure	Key number 0–127	Pressure 0–127	3
0xB _n	Control Change	Controller 0–119	Value 0–127	3
0xB _n	Channel Mode	Controller 120–127	See section 5.3	3
0xC _n	Program Change	Program 0–127	—	2
0xD _n	Channel Pressure	Pressure 0–127	—	2
0xE _n	Pitch Bend Change	Value LSB (7 bits)	Value MSB (7 bits)	3

4.1 — Note numbers and velocity

Key number 60 is middle C, and the octave convention almost everybody uses calls it C4 — so `note = 12 × (octave + 1) + semitone`, with semitone 0 = C. Key 69 is A4, 440 Hz by convention. The usable range is 0 (C-1) to 127 (G9), which is wider than any acoustic instrument and narrower than the audible spectrum.

Velocity is 1–127 for a Note On. **Velocity 0 is a Note Off**, and this is not a convention but the specification: it exists so that running status can express a stream of note-ons and note-offs without ever changing the status byte. A sender must therefore never emit a Note On with velocity 0 meaning "silent note", and a receiver must treat it as a release.

4.2 — Pitch bend, and why LSB comes first

Pitch bend is the only 14-bit value in a single channel message. The two data bytes carry 7 bits each, **least significant byte first**, which is the opposite of the order used by 14-bit Control Change pairs — a genuine inconsistency in the standard, and a reliable source of bugs.

```
value = (msb << 7) | lsb      range 0 .. 16383
center = 8192 = 0x2000      no bend
wire:  En  lsb  msb
```

```
Examples on channel 1:
E0 00 40  center, no bend
E0 00 00  full bend down
E0 7F 7F  full bend up
```

How far "full bend" actually is, is not defined by the message — it is the receiver's Pitch Bend Sensitivity, set through RPN 0 (section 6) and defaulting to ± 2 semitones under General MIDI. A sender that needs a known interval must set the sensitivity first, or accept that the result depends on the instrument.

5 — Control Change, and Channel Mode

Control Change (`0xBn`) is the protocol's extension mechanism. Controllers 0–119 are continuous or switch controllers; 120–127 are reserved for Channel Mode messages, which change how the channel behaves rather than how it sounds.

5.1 — Controllers worth knowing

Number	Controller	Notes
0	Bank Select (MSB)	Paired with 32; selects a bank of programs before a Program Change
1	Modulation Wheel	Required by General MIDI
2	Breath Controller	
4	Foot Controller	
5	Portamento Time	
6	Data Entry (MSB)	Value for the currently selected RPN/NRPN
7	Channel Volume	Required by General MIDI
8	Balance	
10	Pan	0 = left, 64 = center, 127 = right. Required by GM
11	Expression	Required by GM. Scales volume without replacing CC 7
32–63	LSB for controllers 0–31	See section 5.2
64	Sustain Pedal (damper)	≤ 63 off, ≥ 64 on. Required by GM
65	Portamento On/Off	
66	Sostenuto	
67	Soft Pedal	
68	Legato Footswitch	
71	Filter Resonance	Sound Controller 2, by convention
74	Brightness / cutoff	Sound Controller 5, by convention
84	Portamento Control	Carries the source note number
91	Effect 1 Depth (reverb)	
93	Effect 3 Depth (chorus)	
98	NRPN (LSB)	Section 6
99	NRPN (MSB)	Section 6
100	RPN (LSB)	Section 6
101	RPN (MSB)	Section 6

5.2 — 14-bit controllers, and why they mostly are not used

Controllers 0–31 may be paired with 32–63 to form a 14-bit value: send the MSB on controller n , then the LSB on controller $n+32$. Here the MSB comes **first**, unlike pitch bend.

In practice, almost nothing outside bank select uses the pairs. Two 3-byte messages cost 1.9 ms on the current loop for one parameter, and most receivers ignore the LSB. If you need smooth control, a single 7-bit controller sent often usually beats a 14-bit pair sent half as often.

5.3 — Channel Mode messages

CC	Message	Effect
120	All Sound Off	Silence immediately, ignoring release and sustain. Value 0.
121	Reset All Controllers	Controllers to their default; notes keep sounding. Value 0.
122	Local Control	0 = off, 127 = on. Disconnects a keyboard from its own sound engine.
123	All Notes Off	Release every note, as if each got a Note Off. Sustain still applies.
124	Omni Mode Off	Respond on the basic channel only. Implies All Notes Off.
125	Omni Mode On	Respond on every channel. Implies All Notes Off.
126	Mono Mode On	Value = number of channels, 0 = as many as there are voices.
127	Poly Mode On	Value 0. Implies All Notes Off.

ALL NOTES OFF IS NOT A PANIC BUTTON

CC 123 honors the sustain pedal: if the receiver thinks the pedal is down, the notes keep sounding. A reliable panic is CC 123, then CC 120, then CC 121 — on *all sixteen* channels. Many sequencers get this wrong and leave one note droning.

6 — Registered and non-registered parameters

The 120 controllers were not enough, so the standard adds a level of indirection: select a parameter number with two Control Changes, then set its value with two more. Registered Parameter Numbers (RPN) are defined by the MIDI Association; Non-Registered (NRPN) are the manufacturer's own.

Selecting and setting an RPN — pitch bend sensitivity of ± 12 semitones, channel 1:

```

B0 65 00    CC 101, RPN MSB = 0
B0 64 00    CC 100, RPN LSB = 0    -> RPN 0 = pitch bend sensitivity
B0 06 0C    CC 6, data MSB = 12    -> 12 semitones
B0 26 00    CC 38, data LSB = 0    -> 0 cents
B0 65 7F    CC 101, RPN MSB = 127
B0 64 7F    CC 100, RPN LSB = 127  -> RPN Null: deselect

```

RPN	Parameter	Data
0x0000	Pitch Bend Sensitivity	MSB = semitones, LSB = cents
0x0001	Channel Fine Tuning	14-bit, center 0x2000 = no detune
0x0002	Channel Coarse Tuning	MSB in semitones, center 64
0x0003	Tuning Program Select	
0x0004	Tuning Bank Select	
0x0005	Modulation Depth Range	Added with General MIDI 2
0x3FFF	RPN Null	Deselects; no data follows

ALWAYS SEND RPN NULL

The selection is sticky. Leave an RPN selected and the next stray CC 6 or CC 38 — from a control surface, from another track — writes into that parameter. Closing with 101/100 = 127/127 costs six bytes and prevents a class of bug that is very hard to see.

7 — System messages

System messages carry no channel. They divide into three families by how a receiver must treat them.

7.1 — System Common

Status	Message	Bytes	Payload
0xF0	System Exclusive start	variable	See 7.3
0xF1	MTC Quarter Frame	2	Message type in bits 6–4, value in bits 3–0
0xF2	Song Position Pointer	3	14-bit, LSB first, in MIDI beats (6 clocks = a sixteenth note)
0xF3	Song Select	2	Song number 0–127
0xF4	undefined	—	Reserved. Skip it.
0xF5	undefined	—	Reserved; used by some gear as a port select
0xF6	Tune Request	1	Analog oscillators, retune yourselves
0xF7	End of Exclusive	1	Terminates 0xF0

All of these **clear running status**.

7.2 — System Real-Time

Status	Message	Meaning
0xF8	Timing Clock	24 per quarter note, always, whatever the tempo
0xF9	undefined	Reserved
0xFA	Start	Play from the beginning
0xFB	Continue	Play from where we stopped, or from the last Song Position
0xFC	Stop	
0xFD	undefined	Reserved
0xFE	Active Sensing	"I am still here", every 300 ms at most
0xFF	System Reset	Return to power-on state. Almost never sent on purpose.

These are one byte each, they may appear **anywhere** — even between two data bytes of another message — and they do **not** clear running status. That is the whole point: a clock has to be able to arrive on time regardless of what else is being transmitted.

ACTIVE SENSING IS OPT-IN, AND STICKY

A receiver that has seen one 0xFE is entitled to assume the sender will keep sending them, and to silence all notes if none arrives within about 300 ms. So a device that sends Active Sensing must keep sending it. Sending it once, or occasionally, is worse than never sending it at all.

7.3 — System Exclusive

```
F0 <manufacturer ID> <data ...> F7
```

Manufacturer ID is one byte, or three when the first is 0x00:

0x01	Sequential Circuits	0x41	Roland
0x43	Yamaha	0x47	Akai
0x00 0x20 0x33 (example 3-byte form, allocated by the MIDI Association)			

Three reserved IDs matter to anyone writing tools:

0x7D	non-commercial: research, education, private use
0x7E	universal non-real-time
0x7F	universal real-time

Data bytes inside a SysEx block must have bit 7 clear — the payload is 7-bit, so binary data has to be packed, commonly seven bytes into eight. The block is terminated by 0xF7, and a receiver must also abandon it if any other status byte arrives first.

The one universal message worth implementing is Device Inquiry:

```
Request:  F0 7E <device ID> 06 01 F7
Reply:    F0 7E <device ID> 06 02 <manufacturer> <family> <member>
          <version 4 bytes> F7
```

<device ID> is 0x7F for "all devices".

8 — Timing and synchronization

8.1 — MIDI Clock

MIDI Clock is 24 pulses per quarter note (ppqn), a constant. Tempo is expressed by how fast the pulses arrive, and nothing in the stream states the tempo in beats per minute — the receiver infers it, which is why locking to external MIDI clock always has a settling time.

```
interval between clocks = 60 / (BPM × 24) seconds

120 BPM  -> 20.8 ms      (48 clocks per second)
60 BPM   -> 41.7 ms
240 BPM  -> 10.4 ms
```

At 120 BPM the clock alone consumes 48 bytes per second out of roughly 3,125 — about 1.5% of the link. That is negligible; what is not negligible is that each clock byte can delay a note by up to 320 µs.

8.2 — Song Position Pointer

Song Position counts *MIDI beats*, where one beat is six clocks — that is, a sixteenth note. It is 14-bit, so it addresses 16,384 sixteenths: 1,024 bars of 4/4, about 34 minutes at 120 BPM. Beyond that, it wraps, and the usual approach is to not rely on it for long-form material.

The sequence for locating is: `0xFC` Stop, `0xF2` Song Position, then `0xFB` Continue.

8.3 — MIDI Time Code

MTC carries SMPTE timecode. A full frame arrives as eight Quarter Frame messages (`0xF1`), each holding a 4-bit nibble and a 3-bit selector — so two frames' worth of messages are needed to transmit one complete timecode value, and the reader is always a frame or two behind. A full timecode value can also be sent as a universal real-time SysEx, which is what a jump uses.

PHYTOSCOPE

The software sends no clock, no Song Position and no MTC. A plant is not a metronome: events arrive when they arrive, and imposing a grid on them would be a musical decision presented as a measurement. Section 12.

9 — Standard MIDI Files

An SMF is a container of chunks, each with a 4-character type and a 32-bit big-endian length. Two types exist: one header, then one or more tracks.

9.1 — The header chunk

```
"MThd" 00 00 00 06 <format:2> <ntrks:2> <division:2>

format    0 one track containing everything
          1 several tracks, played simultaneously – the common case
          2 several tracks, each an independent sequence

ntrks     number of MTrk chunks that follow

division  if bit 15 = 0: ticks per quarter note (e.g. 480, 960)
          if bit 15 = 1: SMPTE – upper byte is the negative frame rate
                           (-24, -25, -29, -30), lower byte is ticks
                           per frame
```

9.2 — Track chunks and delta times

```
"MTrk" <length:4> then a stream of: <delta-time:VLQ> <event>
```

Delta times are variable-length quantities: seven bits of payload per byte, with bit 7 set on every byte except the last. This is the same encoding as Git's varints and Protocol Buffers', and it is the reason an SMF can hold a long silence in two bytes.

```
VLQ encoding
  0      -> 00
  64     -> 40
  127    -> 7F
  128    -> 81 00
  8192   -> C0 00
  1048576 -> C0 80 00
  0x0FFFFFFF -> FF FF FF 7F (the maximum a VLQ may hold)
```

Running status applies inside a track, and MIDI files use it heavily. The status byte is remembered across delta times, which is exactly where a naive reader goes wrong: the delta time is not a status byte, and it must not reset the running status.

9.3 — Meta events

Meta events are `0xFF <type> <length:VLQ> <data>` and never reach the wire — they exist only in files.

Type	Meta event	Data
0x00	Sequence Number	2 bytes
0x01	Text	Any text
0x02	Copyright Notice	Should be the first event of the first track
0x03	Track Name	Sequence name in track 0 of a format 1 file
0x04	Instrument Name	
0x05	Lyric	One syllable, usually

Type	Meta event	Data
0x06	Marker	A named point in time
0x07	Cue Point	
0x20	MIDI Channel Prefix	1 byte; applies to following meta events
0x21	MIDI Port	1 byte; non-standard but widespread
0x2F	End of Track	Length 0. Required , and must be last.
0x51	Set Tempo	3 bytes: microseconds per quarter note
0x54	SMPTE Offset	5 bytes: hr mn se fr ff
0x58	Time Signature	4 bytes: nn dd cc bb — see below
0x59	Key Signature	2 bytes: sf (−7..+7), mi (0 major, 1 minor)
0x7F	Sequencer Specific	Manufacturer ID, then anything

Set Tempo — 120 BPM

$60,000,000 / 120 = 500,000 \mu\text{s}$ per quarter = 0x07A120

FF 51 03 07 A1 20

Time Signature — 6/8, metronome every dotted quarter, 24 MIDI clocks per quarter note

FF 58 04 06 03 24 08

```

      | | | |
      | | | +-- bb: 32nd notes per quarter (almost always 8)
      | | +----- cc: MIDI clocks per metronome click
      | +----- dd: denominator as a power of two (3 -> 8)
      +----- nn: numerator

```

TICKS ARE NOT TIME

A tick is a fraction of a quarter note, and the quarter note's duration changes at every Set Tempo. Converting a tick position to seconds requires walking the tempo map from the start of the file. Code that multiplies ticks by a single tempo is correct only for files that never change tempo — and it fails silently on the ones that do.

10 — General MIDI

MIDI says nothing about what a program number sounds like. General MIDI Level 1 fixes that by assigning 128 sounds to the 128 program numbers, and by requiring a minimum capability: 24 simultaneous voices, all sixteen channels, channel 10 for percussion, and support for controllers 1, 7, 10, 11, 64, 121 and 123 plus RPN 0, 1 and 2, with pitch bend defaulting to ± 2 semitones.

10.1 — Program families

The 128 programs are grouped in sixteen families of eight. Program numbers here are **1-based**, as GM documents them; the byte on the wire is one less.

Programs	Family	Programs	Family
1–8	Piano	65–72	Reed
9–16	Chromatic Percussion	73–80	Pipe
17–24	Organ	81–88	Synth Lead
25–32	Guitar	89–96	Synth Pad
33–40	Bass	97–104	Synth Effects
41–48	Strings	105–112	Ethnic
49–56	Ensemble	113–120	Percussive
57–64	Brass	121–128	Sound Effects

10.2 — The percussion key map on channel 10

On channel 10 the key number selects an instrument, not a pitch.

Key	Instrument	Key	Instrument	Key	Instrument
35	Acoustic Bass Drum	51	Ride Cymbal 1	67	High Agogo
36	Bass Drum 1	52	Chinese Cymbal	68	Low Agogo
37	Side Stick	53	Ride Bell	69	Cabasa
38	Acoustic Snare	54	Tambourine	70	Maracas
39	Hand Clap	55	Splash Cymbal	71	Short Whistle
40	Electric Snare	56	Cowbell	72	Long Whistle
41	Low Floor Tom	57	Crash Cymbal 2	73	Short Guiro
42	Closed Hi-Hat	58	Vibraslap	74	Long Guiro
43	High Floor Tom	59	Ride Cymbal 2	75	Claves
44	Pedal Hi-Hat	60	Hi Bongo	76	Hi Wood Block
45	Low Tom	61	Low Bongo	77	Low Wood Block
46	Open Hi-Hat	62	Mute Hi Conga	78	Mute Cuica
47	Low-Mid Tom	63	Open Hi Conga	79	Open Cuica
48	Hi-Mid Tom	64	Low Conga	80	Mute Triangle
49	Crash Cymbal 1	65	High Timbale	81	Open Triangle
50	High Tom	66	Low Timbale		

10.3 — GM2, GS and XG

General MIDI 2 extends GM1 with bank select, more controllers, more programs and a defined reverb and chorus. Roland's GS and Yamaha's XG are earlier, incompatible extensions of the same idea. A sender that wants to work everywhere sticks to GM1, sends a GM1 System Exclusive reset if it needs a known state, and treats anything beyond as a bonus.

```
GM1 System On — universal non-real-time SysEx  
F0 7E 7F 09 01 F7
```

11 — MIDI 2.0, UMP and MIDI-CI

MIDI 2.0 is not a replacement but an envelope. Its transport unit is the Universal MIDI Packet (UMP): 32, 64, 96 or 128 bits, whose first four bits say which message type it holds — and one of those types is MIDI 1.0, unchanged. A MIDI 2.0 device therefore speaks MIDI 1.0 natively, and everything in sections 3 to 8 keeps applying.

Type	Content	Size
0x0	Utility — NOOP, jitter-reduction timestamps	32 bits
0x1	System Real-Time and System Common	32 bits
0x2	MIDI 1.0 Channel Voice	32 bits
0x3	Data — 7-bit SysEx	64 bits
0x4	MIDI 2.0 Channel Voice	64 bits
0x5	Data — 8-bit SysEx, mixed data set	128 bits

What the 64-bit channel voice messages buy:

- **Resolution.** Velocity becomes 16-bit, controllers 32-bit. The 7-bit steps that make a slow filter sweep audibly staircase are gone.
- **Per-note controllers and per-note pitch bend.** In MIDI 1.0, pitch bend belongs to the channel, so bending one note of a chord means putting it on its own channel. MIDI 2.0 addresses the note.
- **Groups.** Sixteen groups of sixteen channels — 256 channels on one connection.
- **Registered controllers** that no longer need the four-CC dance of section 6.

MIDI-CI (Capability Inquiry) is the negotiation layer, carried over MIDI 1.0 SysEx so that it works on existing cables. Three things it does: Protocol Negotiation (agree to switch to MIDI 2.0), Profile Configuration (a device declares "I am a drawbar organ" and a known controller layout follows), and Property Exchange (JSON over SysEx for names, patch lists, state).

PHYTOSCOPE

The software emits MIDI 1.0 only. There is a real reason to want MIDI 2.0 here — 7-bit velocity quantizes a measured amplitude into 127 steps, and the measurement deserves better — but no reason to want it before the receiving software does. It is a candidate for a later version, recorded in `.ai/decisions.md` rather than half-implemented.

12 — What PhytoScope emits, and what it does not

Measured from `phytoscope/music/midi_out.py`, version 1.5.1. Four message types, and nothing else.

Status	Message	How PhytoScope uses it
0x90	Note On	One per detected event. Velocity clamped to 1-127, so a Note On is never mistaken for a release.
0x80	Note Off	Scheduled automatically at the note's duration, and sent with release velocity 0.
0xB0	Control Change	Available to modules through the API; the software itself sends none by default.
0xC0	Program Change	Once, when the user picks an instrument.

12.1 — The channel off-by-one, made explicit

The public API takes a channel numbered 1-16, as a musician says it, and converts once:

```
ch = (channel - 1) & 0x0F
self._send(NOTE_ON | ch, midi, max(1, min(velocity, 127)))
```

Doing the subtraction in exactly one place is deliberate. The alternative — 0-based at the boundary, 1-based in the interface — produces a bug that only shows up on channel 16 wrapping to channel 1, which is the kind of thing that is found by a user and not by a test.

12.2 — Every data byte is masked

```
self._port.send_message([status & 0xFF, d1 & 0x7F, d2 & 0x7F])
```

The `& 0x7F` is not defensive clutter. A data byte with bit 7 set is a status byte, so a single out-of-range value does not produce a wrong note — it produces a stream the receiver resynchronizes from, losing the messages in between. Masking converts a protocol violation into a wrong value, which is a much smaller failure.

12.3 — What is deliberately absent

No running status

Full status bytes on every message. The bandwidth saving is real but small at the rates a plant produces, and a stream that any parser reads correctly is worth more than 33% of a link we are not saturating.

No clock, no Song Position, no MTC

Events happen when the plant produces them. Emitting a tempo would be asserting a musical grid that the measurement does not contain.

No pitch bend

Its range depends on the receiver's Pitch Bend Sensitivity, so the same stream would sound different on two synthesizers. A mapping whose result is not reproducible is not a measurement.

No System Exclusive

Nothing to say that is not already in the note stream, and SysEx is by definition device-specific.

No Active Sensing

Section 7.2: sending it once obliges you to keep sending it. The software has no need, so it never starts.

13 — Pitfalls that cost real time

Note On with velocity 0 is a Note Off

Not a convention — the specification. A "silent note" sent this way releases a note instead.

Pitch bend sends LSB first; 14-bit CC pairs send MSB first

An inconsistency in the standard, and the reason a bend that should go up goes 128 times too far.

System Real-Time bytes appear inside other messages

A clock can land between a note number and its velocity. A parser that is not written for it loses notes whenever the sender is synchronized.

All Notes Off obeys the sustain pedal

A stuck note survives CC 123. Panic needs 123, then 120, then 121, on all sixteen channels.

An RPN stays selected

Close with RPN Null (101/100 = 127/127) or the next CC 6 from anywhere writes into your parameter.

SMF ticks are not time

Walk the tempo map. A single multiplication is right only until the first Set Tempo.

End of Track is mandatory

A track without FF 2F 00 is malformed, and readers disagree about what to do with it.

320 µs per byte

A ten-note chord is 10 ms wide on a DIN cable. If the timing matters, measure it rather than assuming the port is instant.

Channel 10 is percussion, and only under General MIDI

On a non-GM instrument it is an ordinary channel. Do not send a drum map to something that never agreed to one.

14 — Normative references

These are the documents that decide. They are published by the MIDI Association and are copyrighted; this repository does not redistribute them, which is why you are reading a summary and not a copy.

MIDI 1.0 Detailed Specification, version 4.2.1

The message set, the electrical layer, the timing. Everything in sections 2 to 8 is a condensation of this.

Standard MIDI Files 1.0 (SMF)

The chunk format, delta times, meta events. Section 9.

General MIDI System Level 1, and General MIDI 2

Program assignments, the percussion key map, minimum capability. Section 10.

MIDI 2.0 Specification, and the UMP and MIDI-CI documents

Universal MIDI Packet, MIDI 2.0 Channel Voice messages, Capability Inquiry, Profiles, Property Exchange. Section 11.

midi.org — The MIDI Association

Where the specifications are obtained, and where manufacturer identifiers are allocated.

14.1 — Provenance of this document

Written for PhytoScope in September 2026, from the specifications above and from what the project's own MIDI output does. The figures in section 12 were read from the source, not remembered: `NOTE_ON = 0x90`, `NOTE_OFF = 0x80`, `PROGRAM_CHANGE = 0xC0`, `CONTROL_CHANGE = 0xB0`, velocity clamped to 1-127, channel converted once as `(channel - 1) & 0x0F`.

This file and its stylesheet are MIT licensed, like the rest of the software. It is regenerated by `sources/midi/build.py`; edit the HTML, never the PDF.